

Concord: A Collaborative Code Execution Platform

CS455: Introduction to Software Engineering · Instructor: Prof. Priyanka Bagade

Deliverable 0: Project Proposal

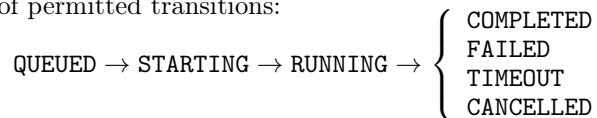
Pallav Rastogi (230731) · Aamir Ahmad (230010) · Aryamann Srivastava (230211) · Tattwa Shivani (231089) · Sanchit Arora (230907)

1. Description. Concord is a web platform where several users join a shared room and edit the same source file at the same time, with live cursors and presence. Any member can run the file in an isolated Docker sandbox, and the output goes to everyone in the session rather than only the person who started the run. We plan to support Python, C++ and JavaScript. An AI assistant sits beside the editor, answers questions about the open file, and can rewrite it in place.

2. User roles. The role rank is `viewer < editor < owner`. We check it on every REST endpoint that changes state, and on every incoming WebSocket edit message rather than only at connection time.

Role	Can do
Viewer	Join a session, read the document as it changes, see other members' cursors and presence, view execution output, and ask the AI assistant questions. Cannot change anything.
Editor	Everything a viewer can do, plus edit the document, start and cancel runs, apply AI generated code, and create or restore snapshots.
Owner	Everything an editor can do, plus create the session, add / remove members, change roles, close the session.

3. Core transactional workflow. Running code is the main transaction in the system. We model it as a state machine with an explicit table of permitted transitions:



An editor presses Run. The backend reads the latest in-memory document, creates an `executions` row as `QUEUED`, and pushes a job to Redis. A worker moves it through `STARTING` and `RUNNING`, then launches a language-specific container. C++ has a compile step first; if compilation fails we report the compiler error and never run the program. The worker captures stdout, stderr, exit code and elapsed time. The final status and output are published on Redis and forwarded over WebSocket to every member of the session. Each status update is validated against the transition table, so nothing can move a finished run back into `RUNNING`.

4. Shared resource and the concurrency challenge. The contended resource is the document itself, since several people type into the same file at once. A last write wins approach loses edits and is not suitable.

We plan to use a CRDT (Conflict-free Replicated Data Type) for this, specifically Yjs. Each character gets a stable identity derived from the client that created it and a logical clock, rather than a numeric position. Two replicas that receive the same edits in any order therefore end up with the same text.

The execution queue is a smaller second point of contention, since more than one member may request a run at once. Redis turns these into a single queue that the workers draw from.

5. AI agent: placement, tools and rationale. One agent runs in a panel to the right of the editor, scoped to one session and one file. It runs entirely on the backend, so the browser never holds the API key and permissions, rate limiting and logging all have a single place to live. We give it two tools:

Tool	Purpose
<code>read_active_document</code>	Returns the current contents of the open file, so the agent works from what the user is looking at, including edits others made a moment ago.
<code>replace_active_document</code>	Writes a full replacement into the shared document.

6. Technology stack. React, TypeScript and Vite with the Monaco editor on the frontend. Yjs for the CRDT with the `y-monaco` binding over WebSockets. FastAPI with SQLAlchemy and Alembic on the backend, PostgreSQL for storage, Redis for the queue and pub/sub. Each run gets its own Docker container with no network, capped memory, CPU and process count, a read-only filesystem and a non-root user, using one image per language (`python:3.12-slim`, `node:22-slim`, `gcc:13`). Authentication is JWT with bcrypt. CI runs on GitHub Actions and deployment is Docker Compose on a cloud VM behind a TLS reverse proxy.

7. Risks and technical uncertainties.

- **Real-time collaboration:** Failure here would break the core feature, making it the highest-risk component.
- **CRDT persistence:** Binary document and state-vector data can fail silently, risking data loss or inconsistent restores.
- **Execution and AI reliability:** The sandbox must securely handle untrusted code, while AI rate limits, invalid code output, and unverified performance targets may affect reliability.

8. GitHub and Jira strategy.

Repository: GitHub Repo

Jira board: Jira Board